Security Audit Report

MANTRA RWA Primary Sale Smart Contract (ID RWA-01)

v1.0

January 12, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 8
Functionality Overview 8
How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 13
1. MAX_INVESTORS limit is declared but never enforced during investment 13
2. Missing setter function for accepted tokens 13
3. Inefficient external call for asset token decimals during batch settlement 14
4. Restricted and delisted investors' contributions count toward SOFT_CAP but are refunded during settlement, potentially leaving the issuer below the threshold 15
5. Self-transfer for a restricted investor refund may cause settlement failure 16
6. Centralization risks 16
7. Push payment pattern vulnerable to DoS via token blacklist mechanisms 19
8. Excessive gas consumption in the settleBatch function due to unnecessary calculations 20
9. Zero-value settlement transfers may revert for restrictive ERC20 tokens 23
10. Possibility of locking leftover funds in the contract added via the topUpRefunds function 23
11. The settleBatch function can still be called after the sale has been completed 24
12. MIN_STEP is not validated to be an 18-decimal value 24
13. Mixed-decimal totalRefund value 25
14. Refund path for failed or cancelled sale ignores the allowlist and lets restricted users withdraw 25
15. Asset owner token spending allowance can block settlement and force manual refunds 26
16. No balance check when forwarding invested tokens may break with fee-on-transfer tokens 26
17. Suboptimal storage packing in SettlementState struct increases overflow risk 27
18. Floating pragma version may lead to unexpected compilation behaviour 28
19. Unused onlyFailed modifier should be removed 28
20. Using magic numbers decreases code maintainability 29
21. BatchSettled event misreports batch numbers when batch sizes differ 30
22. Checks-Effects-Interactions pattern not followed in the codebase 30

23. Misleading multisig-specific error in permissionless refund top-up 31
24. Repeated base amount calculations decrease code readability 32
25. DEFAULT_ADMIN_ROLE is unused, and ADMIN_ROLE becomes the de facto owner role 32
26. activate does not check that the current time is before END 33
27. Commission is not reflected in documented soft cap semantics 33
28. topUpRefunds design is inconsistent and practically a no-op for external users 34
29. isSaleActive ignores the paused state 35
30. Per-investment forwarding to MULTISIG adds overhead and complicates cancellations 35
31. Outdated comment about miner timestamp manipulation 36
32. Unused SettlementComplete event 36

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by MANTRA Ventures Limited to perform a security audit of MANTRA RWA Primary Sale Smart Contract.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/MANTRA-Finance/mantra-rwa-token-primary-sale-contract-v1
Commit	`74d0cb5ef831d4bdd85cf714d97458027f50c0fd`
Scope	The scope is restricted to the Solidity smart contracts in the `packages/evm/contracts` directory that have been added in the https://github.com/MANTRA-Finance/mantra-rwa-token-primary-sale-contract-v1/pull/11 pull request.
Fixes verified at commit	`be7288c24684ea0c75b16531ec871325f31596a9`

	Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

MANTRA's primary sale Solidity smart contract enables compliance-gated ERC-20 offerings with multi-stablecoin settlement, automated fees and refunds, and immutable safeguards.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Low-Medium**	-
Code readability and clarity	**Medium-High**	-
Level of documentation	**Medium-High**	-
Test coverage	**High**	`hardhat coverage` reports a test line coverage of **94.12%**.

Summary of Findings

No	Description	Severity	Status
1	`MAX_INVESTORS` limit is declared but never enforced during investment	Minor	Resolved
2	Missing setter function for accepted tokens	Minor	Resolved
3	Inefficient external call for asset token decimals during batch settlement	Minor	Resolved
4	Restricted and delisted investors' contributions count toward `SOFT_CAP` but are refunded during settlement, potentially leaving the issuer below the threshold	Minor	Resolved
5	Self-transfer for a restricted investor refund may cause settlement failure	Minor	Resolved
6	Centralization risks	Minor	Acknowledged
7	Push payment pattern vulnerable to DoS via token blacklist mechanisms	Minor	Acknowledged
8	Excessive gas consumption in the `settleBatch` function due to unnecessary calculations	Minor	Acknowledged
9	Zero-value settlement transfers may revert for restrictive ERC20 tokens	Minor	Resolved
10	Possibility of locking leftover funds in the contract added via the `topUpRefunds` function	Minor	Resolved
11	The `settleBatch` function can still be called after the sale has been completed	Minor	Resolved
12	`MIN_STEP` is not validated to be an 18-decimal value	Minor	Resolved
13	Mixed-decimal `totalRefund` value	Minor	Resolved
14	Refund path for failed or cancelled sale ignores the allowlist and lets restricted users withdraw	Minor	Resolved
15	Asset owner token spending allowance can block settlement and force manual refunds	Minor	Acknowledged
16	No balance check when forwarding invested tokens may break with fee-on-transfer tokens	Minor	Resolved

17	Suboptimal storage packing in `SettlementState` struct increases overflow risk	**Informational**	**Resolved**
18	Floating pragma version may lead to unexpected compilation behaviour	**Informational**	**Resolved**
19	Unused `onlyFailed` modifier should be removed	**Informational**	**Resolved**
20	Using magic numbers decreases code maintainability	**Informational**	**Acknowledged**
21	`BatchSettled` event misreports batch numbers when batch sizes differ	**Informational**	**Resolved**
22	Checks-Effects-Interactions pattern not followed in the codebase	**Informational**	**Resolved**
23	Misleading multisig-specific error in permissionless refund top-up	**Informational**	**Acknowledged**
24	Repeated base amount calculations decrease code readability	**Informational**	**Acknowledged**
25	`DEFAULT_ADMIN_ROLE` is unused, and `ADMIN_ROLE` becomes the de facto owner role	**Informational**	**Acknowledged**
26	activate does not check that the current time is before `END`	**Informational**	**Acknowledged**
27	Commission is not reflected in documented soft cap semantics	**Informational**	**Acknowledged**
28	`topUpRefunds` design is inconsistent and practically a no-op for external users	**Informational**	**Acknowledged**
29	`isSaleActive` ignores the paused state	**Informational**	**Resolved**
30	Per-investment forwarding to `MULTISIG` adds overhead and complicates cancellations	**Informational**	**Acknowledged**
31	Outdated comment about miner timestamp manipulation	**Informational**	**Resolved**
32	Unused `SettlementComplete` event	**Informational**	**Resolved**

Detailed Findings

1. `MAX_INVESTORS` limit is declared but never enforced during investment

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:112`, the contract declares `MAX_INVESTORS = 500` as a constant, and defines the `TooManyInvestors` error in line `69`. However, this limit is never enforced when new investors are added to the investors array.

The `invest` function implemented in lines `609-661` adds new investors to the array in line `644` when `contributedNormalized[msg.sender] == 0`, indicating a first-time investor. The code pushes the investor address without checking whether `investors.length < MAX_INVESTORS`, allowing the array to grow beyond the intended 500-investor limit.

This creates a discrepancy between the declared design constraint and the actual behavior. While the settlement logic includes comments suggesting safety up to 500 investors (lines `1066`, `1070`), exceeding this limit could lead to:

The `TooManyInvestors` error exists but is never thrown, indicating this check was intended but not implemented.

Recommendation

We recommend either removing `MAX_INVESTORS`, or enforcing it in the code.

Status: Resolved

2. Missing setter function for accepted tokens

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:468-485`, the constructor saves `acceptedTokens` and `tokenDecimals` once, and the `PrimarySale` contract does not provide any admin function to remove or disable an accepted token via `isAcceptedToken`.

This means that if an accepted stablecoin depegs before or during the sale, the team cannot update the token set on-chain and must instead pause or cancel and redeploy, or accept that investors can use the depegged asset.

Recommendation

We recommend adding an admin-only function, for example, gated by the `ADMIN_ROLE`, that allows disabling specific payment tokens by updating `isAcceptedToken[token]`.

Status: Resolved

3. Inefficient external call for asset token decimals during batch settlement

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:573`, the `_denormalizeAmount` function makes an external call to `IERC20Metadata(token).decimals` function on every invocation. This differs from `_normalizeAmount` in line `542`, which uses the cached `tokenDecimals` mapping for a storage read.

During batch settlement in `settleBatch` in line `1022`, `_denormalizeAmount` is called for each investor in the loop, causing repeated external calls for the same asset token decimals. Since the asset token is set once during `initializeSettlement` (line `765`) and does not change during settlement, these repeated calls are unnecessary.

The impact is increased gas costs during batch settlement. With up to 100 investors per batch (`MAX_BATCH_SIZE`), this can add significant overhead. Each external call costs approximately 2600 gas (cold) or 100 gas (warm), and additional gas used by the external call logic. For a full batch of 100 investors, this can accumulate to approximately 102500 per batch.

Recommendation

We recommend caching the asset token decimals during `initializeSettlement` and reusing the cached value in `_denormalizeAmount`. The value can be read once at the beginning of the loop, and then passed as an argument to the function.

Alternatively, if the asset token is known beforehand, it can be added in the constructor, and the token's decimals can be specified as an immutable value.

Status: Resolved

4. Restricted and delisted investors' contributions count toward `SOFT_CAP` but are refunded during settlement, potentially leaving the issuer below the threshold

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol`, the `SOFT_CAP` check in the `endSale` function (line `673`) and `initializeSettlement` (line `761`) uses `totalContributedNormalized`, which includes contributions from all investors, including those who will be refunded during settlement.

During settlement in the `settleBatch` function (lines `1030-1041`), restricted wallets and delisted investors are refunded. Restricted wallets have their payment tokens returned to `MULTISIG`, and delisted investors receive their payment tokens back. In both cases, the issuer does not receive these funds, and the refunded amounts are tracked in `settlementState.totalRefundedNormalized`.

The issue is that `totalContributedNormalized` includes contributions from investors who will later be refunded. If many investors are delisted or restricted during settlement, the net amount the issuer receives (`totalContributedNormalized - settlementState.totalRefundedNormalized`) may fall below `SOFT_CAP`, even though the sale passed the `SOFT_CAP` check at `endSale`.

This undermines the `SOFT_CAP` requirement. In scenarios where the sale falls short of `SOFT_CAP`, and many investors are subsequently delisted or restricted, the issuer may receive significantly less funding than intended, thereby defeating the purpose of the minimum funding threshold. For token issuers, this could result in insufficient fundraising while still distributing tokens to a small number of investors, which may not align with their distribution goals.

Recommendation

We recommend redesigning the settlement and token-claiming processes. For instance, during settlement, all investors can be processed and their status set to successful, delisted, or restricted without immediately distributing the tokens. This helps determine the actual sale volume and ensures the `SOFT_CAP` is reached. If enough funds are raised, the sale is marked as successful, and investors can either manually claim their assets or have them distributed by the admin through a new function. Otherwise, the sale is marked as failed, and all investors can be refunded.

Status: Resolved

5. Self-transfer for a restricted investor refund may cause settlement failure

Severity: Minor

The `_refundInvestorTokens` function in `packages/evm/contracts/PrimarySale.sol:810-833` unconditionally executes `safeTransferFrom(MULTISIG, recipient, amount)` in line `827`, regardless of whether the sender and recipient addresses are identical.

When handling restricted investors, the `_handleRestrictedInvestor` function at `packages/evm/contracts/PrimarySale.sol:842-859` invokes `_refundInvestorTokens(investor, MULTISIG)`, passing `MULTISIG` as both the source (hardcoded in the transfer call) and the recipient parameter. This results in a self-transfer operation: `safeTransferFrom(MULTISIG, MULTISIG, amount)`.

While the ERC20 standard does not explicitly prohibit self-transfers, some token implementations revert when `from == to` to prevent unnecessary state changes or to enforce stricter transfer semantics. If any of the accepted payment tokens in the `acceptedTokens` array implement such a restriction, the settlement process will revert when attempting to process restricted investors.

This vulnerability can cause the entire `settleInvestorsInBatches` operation to fail, preventing settlement completion and potentially locking funds in the contract. The issue is particularly problematic because it only manifests during settlement when restricted investors are present, making it difficult to detect during testing if restricted investor scenarios are not thoroughly covered.

Recommendation

We recommend modifying `_refundInvestorTokens` to skip the `safeTransferFrom` call when `recipient == MULTISIG`, since the funds are already held at the `MULTISIG` address. The function should only update the accounting state (`refundedDuringSettlement`) in this case, as no actual token movement is required.

Status: Resolved

6. Centralization risks

Severity: Minor

The settlement mechanism in `packages/evm/contracts/PrimarySale.sol` contains multiple centralization risks that grant the admin excessive control over fund distribution, asset token selection, and investor access to their tokens. The contract lacks safeguards and pull-based mechanisms that would protect investors from admin manipulation or operational failures.

The following issues have been identified:

- Admin can selectively deny presale amounts by restricting accounts during settlement. The `settleBatch` function located in `packages/evm/contracts/PrimarySale.sol:908-1077` accepts a `restrictedWallets` parameter that allows the admin to flag specific investor addresses. When an investor is marked as restricted in `_handleRestrictedInvestor` at `packages/evm/contracts/PrimarySale.sol:842-859`, their payment tokens are refunded to `MULTISIG` instead of the investor, and they receive no asset tokens. The admin can abuse this mechanism by adding any investor to the restricted list during settlement, effectively denying them their presale allocation while keeping their funds.
- The admin can change the primary sale asset token after investors have paid: The asset token is only set during `initializeSettlement` at `packages/evm/contracts/PrimarySale.sol:753-796`, which occurs after the sale has ended and all investors have already contributed their funds. The admin can call this function with any `assetToken` address, potentially distributing worthless or malicious tokens to investors who have already paid with legitimate stablecoins. There is no mechanism to lock or commit to the asset token before the sale begins.
- Admin can rugpull via `emergencyWithdrawERC20` after cancelling the sale: The `emergencyWithdrawERC20` function at `packages/evm/contracts/PrimarySale.sol:1240-1253` can only be called when `status == Status.Cancelled` and allows the admin to withdraw any ERC20 tokens from the contract. Combined with the `cancel` function at `packages/evm/contracts/PrimarySale.sol:1204-1209`, the admin can cancel an active sale and immediately drain all investor funds without any time delay. Investors have no window to withdraw their funds via `claimRefund` before the admin can execute the emergency withdrawal.
- Admin can initialize settlement with an incorrect or malicious asset token: The `initializeSettlement` function at `packages/evm/contracts/PrimarySale.sol:753-796` accepts an `assetOwner` parameter that can be any address. The admin can provide an `assetOwner` address that temporarily holds the required tokens, initialize settlement to pull tokens into the contract, and then the asset owner can move or drain the tokens before settlement completes. Alternatively, the admin can provide an `assetOwner` address not controlled by Mantra, potentially leading to the distribution of unauthorized tokens.
- Admin can lock all user funds by not calling `settleBatch`: The `settleBatch` function in `packages/evm/contracts/PrimarySale.sol:908-1077` can only be called by `ADMIN_ROLE`, and there is no pull-based claim function for investors. Once settlement is initialized, all asset tokens are locked in the contract, but distribution depends entirely on the admin repeatedly calling `settleBatch`. If the admin fails to call this function for all investors (due to key loss, DDOS attacks, technical issues, or malicious intent), later investors' funds remain permanently locked even though their asset tokens are available in the contract.

- Admin can cause double payment by settling and then cancelling: The admin can call `settleBatch` to distribute asset tokens to investors, then call `cancel` in `packages/evm/contracts/PrimarySale.sol:1204-1209` to change the sale status to Cancelled. This allows investors to claim refunds via `claimRefund` at `packages/evm/contracts/PrimarySale.sol:1171-1198` in addition to having already received their asset tokens, resulting in double payment. The contract does not prevent cancellation after settlement has begun.
- Admin can prevent refunds by not calling `topUpRefunds`: When a sale fails or is cancelled, investors can only claim refunds if the refund pool has been funded via `topUpRefunds` in `packages/evm/contracts/PrimarySale.sol:1147-1164`. While this function is permissionless, the funds must come from somewhere. If the admin (who controls `MULTISIG` where investor funds are held) refuses to call this function or transfer funds to the contract, investors cannot retrieve their contributions even though the sale has failed.
- Restricted account funds go to `MULTISIG` instead of the investor, creating a cash grab scenario: When an investor is marked as restricted during settlement in `_handleRestrictedInvestor` in `packages/evm/contracts/PrimarySale.sol:842-859`, their payment tokens are refunded to MULTISIG (the broker) via `_refundInvestorTokens(investor, MULTISIG)` in line `848`, and they receive no asset tokens. The investor loses both their investment funds and their asset token allocation, while MULTISIG receives the funds. Additionally, `hasReceivedSettlement[investor]` is set to `true`, marking them as settled even though they received nothing. This is described in the `RefundReason.RESTRICTED_WALLET` enum "refund goes to MULTISIG", effectively allowing the admin to seize investor funds by marking them as restricted.
- No pull-based claim mechanism may create unfair distribution: The `settleBatch` function in `packages/evm/contracts/PrimarySale.sol:908-1077` can only be called by ADMIN_ROLE, and there is no pull-claim function for investors. This means the admin decides when each batch runs and how many investors get settled in each call. If the admin runs only the first batch and then stops (due to DDOS attacks, power issues, key loss, or malicious intent), early investors in the investors array get their tokens and can sell them, while later investors stay stuck even though the asset tokens are already locked in the contract. This creates an unfair advantage based solely on array position rather than investment timing, and would not be fair even if the design is intentional.

Recommendation

We recommend implementing the following mitigations to address the centralization risks:

- Add pull-based claim mechanism: Implement a `claimSettlement` function that allows each investor to pull their own tokens once settlement is initialized. This function should check that `settlementState.isInitialized == true`, verify the investor has not already received settlement (`!hasReceivedSettlement[investor]`), validate KYC status via

`ALLOWLIST.isAllowed(investor)`, calculate and transfer the appropriate asset tokens, and update settlement tracking. This would complement the existing `settleBatch` function and allow investors to claim tokens independently.

- Lock the asset token before the sale begins: Add an immutable `assetToken` parameter to the constructor or to a separate initialization function that must be called before the sale activates. This prevents the admin from changing the asset token after investors have contributed funds.
- Add time delay for emergency withdrawals: Modify `emergencyWithdrawERC20` to require a minimum time delay (e.g., 2 weeks) after the sale is cancelled or ended before allowing withdrawals. This gives investors a window to claim refunds via `claimRefund` before the admin can drain funds.
- Prevent cancellation after settlement begins: Add a check in `cancel` to prevent cancellation once `settlementState.isInitialized == true`, preventing the double payment scenario where investors receive both asset tokens and refunds.
- Add safeguards for restricted wallet handling: Consider requiring additional validation or a time delay before marking wallets as restricted, or implement a dispute mechanism. Alternatively, ensure that restricted wallets receive their payment token refunds directly rather than sending funds to `MULTISIG`.
- Validate `assetOwner` in `initializeSettlement`: Add validation to ensure the `assetOwner` is a trusted address (e.g., `ISSUER` or a whitelist) and verify that the asset owner maintains sufficient balance throughout the settlement process.
- Automatic refund mechanism: Consider implementing an automatic refund mechanism that transfers funds from `MULTISIG` to the contract's refund pool when a sale fails or is cancelled, rather than relying on manual `topUpRefunds` calls.

Status: Acknowledged

The client acknowledges this finding, noting that these are the intended superpowers for the admin.

7. Push payment pattern vulnerable to DoS via token blacklist mechanisms

Severity: Minor

The contract uses push payments via `safeTransferFrom` and `safeTransfer` to distribute tokens during settlement and refunds. In `packages/evm/contracts/PrimarySale.sol:827`, `_refundInvestorTokens` transfers payment tokens from MULTISIG to recipients.

If a payment token implements blacklisting (e.g., tokens on Mantra chain, which use a custom ERC20 Cosmos SDK precompile with recipient address blacklisting or USDC), and a recipient is blacklisted, the transfer will revert. This causes a DoS where the entire settlement batch fails, blocking other investors from receiving tokens or refunds. Settlement cannot proceed until the blacklisted address is removed, which may be outside the protocol's control.

Tokens with transfer hooks could also drain gas or cause unexpected behavior when transfers are initiated to recipient addresses, increasing the DoS risk. The push payment pattern exposes the contract to these external token behaviors, preventing completion if any single recipient is unable to receive tokens.

Recommendation

We recommend implementing a pull payment pattern rather than a push payment pattern. Recipients should be able to withdraw their entitled tokens on demand rather than having tokens pushed automatically. This approach would:

Allow settlement processing to complete successfully even if some recipients are temporarily blacklisted

1. Give recipients control over when they claim their tokens
2. Prevent gas-draining attacks from tokens with malicious hooks
3. Make the contract more resilient to external token behaviors

The contract could maintain a mapping of withdrawable balances per recipient and provide a withdrawal function that recipients can call when they are ready to receive their tokens.

Status: Acknowledged

The client acknowledges this finding, noting that the Pull vs Push discussion took place during development. Push was the preferred user experience. In the rare instance where an address is blocked by a token, this can be managed by removing the user from the allowlist.

8. Excessive gas consumption in the `settleBatch` function due to unnecessary calculations

Severity: Minor

The `settleBatch` function in `packages/evm/contracts/PrimarySale.sol:908-1077` performs multiple unnecessary calculations, significantly increasing gas consumption. The function implements a complex state pollution tracking and cleanup mechanism for the `_tempRestrictedMap` function, which is fundamentally unnecessary because cleaning the state in this case does not give any benefit due to the atomic nature of the EVM.

Specifically, the function:

- Implements a pollution tracking system with `_lastRestrictedWallets` array (lines `919-939`, `941-951`) to handle cleanup of stale data from failed transactions, which adds significant gas overhead, and is cleaned at the end of the function.
- Uses a hybrid duplicate detection mechanism with `DUPLICATE_CHECK_THRESHOLD` (lines `970-999`) that is unnecessary, as the worst-case scenario of duplicate addresses in `restrictedWallets` is that awarm SSTORE will be performed for the duplicated address.

- Performs cleanup operations that are self-defeating: `delete _tempRestrictedMap[restrictedWallets[j]]` in lines `1055-1057` is unnecessary since the mapping is temporary, and `delete _lastRestrictedWallets` in line `1062` nullifies the entire cleanup mechanism by clearing the array that was meant to track pollution for subsequent calls
- Checks for the amount of data to clean reaching `MAX_POLLUTION_SIZE` in line `945`, which is unnecessary, because `_lastRestrictedWallets` is deleted on each call to this function

The pollution calculation mechanism attempts to handle edge cases where previous transactions reverted. However, this incurs substantial gas costs (~200K gas overhead for cleanup operations) without providing proportional value. The worst-case scenario of saving a duplicate value to `_tempRestrictedMap` is negligible compared to the gas cost of the cleanup infrastructure.

These inefficiencies make the function unnecessarily expensive to execute, potentially causing transaction failures when gas limits are reached, and increasing operational costs for settlement. Given that the `Allowlist` contract already provides the necessary status information, the entire restricted wallet tracking mechanism (lines `919-999`) can be eliminated.

Additionally, there's a potential operational risk when the caller of the `settleBatch` function fails to add the specific user address to `restrictedWallets` for the current batch, where this address is processed. If it's omitted, it will process the user and settle the investor.

Recommendation

We recommend implementing one of the following solutions to significantly reduce gas consumption:

- Change `Allowlist` to use enum instead of boolean: Modify the `Allowlist` contract to hold an enum (`NotAllowed`, `Restricted`, `Allowed`) instead of a boolean. The current `isAllowed` interface can be maintained for backward compatibility, with a new function added to return the enum value. This would eliminate the need for the `_tempRestrictedMap` mechanism entirely, allowing removal of lines `919-999` and reducing the possibility of bugs in pollution calculation.
- Use bitfield packing for restricted wallets: Instead of passing restricted wallets as an address array, use bitfield packing by encoding each boolean as a single bit in a uint256. Since investor IDs cannot change after block finality, 256 investor flags can be packed into a single array element. Given that no more than 500 investors are expected to participate in the sale, this solution is gas-efficient and straightforward to implement. Exemplary code for bitfield packing is defined below:

```
function setFlag(uint256 storage bitfield, uint256 index, bool value)
internal pure returns (uint256) {
    if (value) {
        return bitfield | (1 << index);
```

```
    } else {
        return bitfield & ~(1 << index);
    }
}

function getFlag(uint256 bitfield, uint256 index) internal pure returns
(bool) {
    return (bitfield & (1 << index)) > 0;
}

// Usage for an array of uint256 flags
function setFlagInArray(uint256[] calldata restrictedWallets, uint256
idx, bool value) internal {
    uint256 arrayIndex = idx / 256;
    uint256 bitIndex = idx % 256;
    restrictedWallets[arrayIndex] =
setFlag(restrictedWallets[arrayIndex], bitIndex, value);
}

function getFlagFromArray(uint256[] calldata restrictedWallets, uint256
idx) internal view returns (bool) {
    uint256 arrayIndex = idx / 256;
    uint256 bitIndex = idx % 256;
    return getFlag(restrictedWallets[arrayIndex], bitIndex);
}
```

- Pass restricted investor indices instead of addresses: Calculate which investor IDs will be restricted off-chain and pass restricted investor indices instead of addresses. Then, instead of writing to and comparing with storage state, simply check if the current investor ID matches the next restricted investor ID. Verify on-chain that the restricted IDs are sorted in ascending order to ensure nothing is missed.

If implementing a partial solution, the following additional optimizations should be considered:

- Remove `DUPLICATE_CHECK_THRESHOLD` logic: Sort restricted addresses off-chain and verify they are sorted in ascending order on-chain
- Remove `delete _tempRestrictedMap[restrictedWallets[j]]` in lines `1055-1057`: This cleanup is unnecessary for a temporary mapping
- Remove `delete _lastRestrictedWallets` in line `1062`: This nullifies the purpose of the cleanup mechanism

Status: Acknowledged

The client acknowledges this finding with the note that the restricted investors list is separate from the allowlist. Furthermore, the allowlist should remain simple for this deployment.

9. Zero-value settlement transfers may revert for restrictive ERC20 tokens

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:1105-1112`, the settlement logic always calls the `paymentTokenContract.safeTransferFrom` function for `tokenCommission` and `tokenIssuerAmount`, even when both evaluate to zero. This occurs when the sale reaches `SOFT_CAP` and every contributed amount is subsequently restricted, leaving `netAmount` equal to the previously refunded quantity. Many ERC20 tokens, especially custodial or non-standard assets, reject zero-value transfers to prevent unnecessary bookkeeping. In such cases, the settlement routine reverts, leaving the entire batch inoperable and trapping funds in the multisig until manual intervention.

Recommendation

We recommend guarding each transfer with a zero check (e.g., skip the call when the amount is zero) or replacing `safeTransferFrom` with a helper that tolerates zero-value transfers. This ensures settlement succeeds even when the net commission and issuer amounts are zero.

Status: Resolved

10. Possibility of locking leftover funds in the contract added via the `topUpRefunds` function

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:1240-1254`, the `emergencyWithdrawERC20` function restricts the execution to only be able to be called when `status == Status.Cancelled`.

However, the sale lifecycle allows `topUpRefunds` to pull additional stablecoins into the contract after most obligations are fulfilled. Once the sale ends successfully, the `Status.Settled` is set, any surplus introduced by `topUpRefunds` remains stranded because `EmergencyWithdrawalNotAllowed` prevents the admin from sweeping the balance to the issuer. These leftover tokens become permanently locked in the contract, forcing a redeploy or governance patch to recover them and creating accounting mismatches between the issuer wallet and the primary-sale contract.

Recommendation

We recommend permitting `emergencyWithdrawERC20` to run when the status is either `Status.Cancelled` or `Status.Settled`. This ensures every surplus introduced by

`topUpRefunds` can be recovered once investor refunds and issuer/treasury distributions are completed.

Status: Resolved

11. The `settleBatch` function can still be called after the sale has been completed

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:908-918`, the `settleBatch` function checks `settlementState.isInitialized`, `settlementState.isComplete`, `batchSize`, and `restrictedWallets.length`, but it does not verify that `settlementState.processedInvestors` is still less than `investors.length`.

This allows an admin to repeatedly call `settleBatch` after all investors have already been processed, performing unnecessary work and emitting additional batch events without changing the settlement state.

Recommendation

We recommend adding a guard, such as `if (settlementState.processedInvestors >= investors.length) revert SettlementAlreadyComplete();` at the start of `settleBatch` to prevent further calls once all investors are processed.

Status: Resolved

12. `MIN_STEP` is not validated to be an 18-decimal value

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:495-499`, `MIN_STEP` is set directly from `minStep_` without enforcing that it is expressed in 18 decimals (for example `minStep_ % 1e18 == 0`), even though comments state it is "provided in 18 decimals".

Later formulas, such as `(totalContributedNormalized / MIN_STEP) * 1e18` assume that `MIN_STEP` represents a clean 18-decimal step, so a non-18-decimal `MIN_STEP` makes these calculations harder to reason about and can introduce subtle rounding behavior.

Recommendation

We recommend adding a constructor check that requires `minStep_ % 1e18 == 0`, or alternatively, explicitly document and handle non-18-decimal values.

Status: Resolved

13. Mixed-decimal `totalRefund` value

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:1189-1192`, `totalRefund` is accumulated as a raw sum of `amount` values across all `acceptedTokens`, even though each token can have different decimals. This mixed sum is then emitted in `RefundClaimed`, so the amount field in the event combines values with different precisions and does not represent a meaningful monetary quantity.

Recommendation

We recommend `normalizing` per-token amounts to a common 18-decimal representation before summing and emitting, or emitting per-token refund amounts instead of a single mixed `totalRefund`.

Status: Resolved

14. Refund path for failed or cancelled sale ignores the allowlist and lets restricted users withdraw

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:1178-1181`, the `claimRefund` function for `failed` or `cancelled` sales only checks `status`, `refunded[msg.sender]`, and pool balances, but does not validate `ALLOWLIST.isAllowed(msg.sender)` or any restricted wallet list.

This means an investor who contributes while allowed and is later `restricted` can still call the `claimRefund` function and recover their funds if the sale `fails` or is `cancelled`. In contrast, the settlement path for successful sales explicitly routes restricted investors through the `_handleRestrictedInvestor` function to send funds to `MULTISIG` instead of the investor. The compliance guarantee that funds of restricted users are confiscated is therefore not consistently enforced across both failure and success paths.

Recommendation

We recommend adding an `allowlist` or `restricted-wallet check` in `claimRefund` so that `restricted` or `de-listed` investors cannot withdraw directly and their refunds are

instead routed to a controlled address (for example, `MULTISIG`), in line with the behavior implemented for `_handleRestrictedInvestor` during settlement.

Status: Resolved

15. Asset owner token spending allowance can block settlement and force manual refunds

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:733-744`, the `_validateSettlementPreconditions` function checks that the `assetOwner` allowance for `totalTokensWithDecimals` is sufficient, and `initializeSettlement` reverts with `InsufficientAssetTokenAllowance` if the owner has reduced or removed this allowance by the time settlement is started.

In that case, settlement cannot proceed, even though investors have already sent funds and the multisig is funded. Therefore, the team must fall back on manual refunds, which are gas-inefficient and operationally slow. Since the asset token requirement is fully determined by `HARD_CAP` and the final `totalContributedNormalized`, only checking and pulling tokens at settlement time introduces unnecessary liveness and gas-spending risk if the owner changes allowance.

Recommendation

We recommend pulling the asset tokens earlier, for example, in the `constructor`, or in `activate`, or otherwise enforcing an earlier allowance check so an `assetOwner` allowance reduction cannot later block settlement.

Status: Acknowledged

The client acknowledges this finding, noting that checking and pulling at settlement is a mandatory requirement.

16. No balance check when forwarding invested tokens may break with fee-on-transfer tokens

Severity: Minor

In `packages/evm/contracts/PrimarySale.sol:634-637`, the `invest` function uses the `amount` parameter for both `safeTransferFrom` into the contract and `safeTransfer` to `MULTISIG` without verifying how many tokens the contract actually received. If a supported token ever implements a transfer fee or otherwise transfers less than the amount to the contract, this can cause forwarding to fail or make the internal accounting inconsistent with the real token flows.

For instance, `USDT` is theoretically a fee-on-transfer token, but the fee implementation is not carried out and is subject to the entity owners.

Recommendation

We recommend using a before/after balance pattern around the `safeTransferFrom` call and forwarding exactly the `received amount` (or reverting if it is less than expected) to remain compatible with potential fee-on-transfer behavior.

Status: Resolved

17. Suboptimal storage packing in `SettlementState` struct increases overflow risk

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:198-214`, the `SettlementState` struct uses inefficient storage packing. `processedInvestors` is `uint128` but only needs to hold up to `MAX_INVESTORS` (`500`), so `uint16` (max `65535`) is sufficient. `totalDistributedTokens` is also `uint128`, which limits its range and increases the risk of overflow during large token distributions.

Currently, the struct uses three storage slots: one for the boolean flags and asset token, a second for `processedInvestors` and `totalDistributedTokens`, and a third for `totalRefundedNormalized`. Using `uint16` for processedInvestors and `uint240` for `totalDistributedTokens` would keep the packing of both in a single 256-bit slot, while providing a larger range for `totalDistributedTokens`, reducing overflow risk.

Recommendation

We recommend changing `processedInvestors` from `uint128` to `uint16` and `totalDistributedTokens` from `uint128` to `uint240` in the `SettlementState` struct.

This requires updating explicit casts to `uint128` in `packages/evm/contracts/PrimarySale.sol:1067` and `1047` to use the appropriate types (`uint16` and `uint240`, respectively). This optimization maintains functionality while improving gas efficiency and reducing overflow risk.

Status: Resolved

18. Floating pragma version may lead to unexpected compilation behaviour

Severity: Informational

In `packages/evm/contracts/Allowlist.sol:2` and `packages/evm/contracts/PrimarySale.sol:2`, the pragma directive uses a floating version range (*^0.8.24*) instead of a fixed version. The ^ allows any version from *0.8.24* up to (but not including) *0.9.0*, which can cause inconsistent compilation across environments and introduce unexpected behavior from newer compiler versions.

Recommendation

We recommend using a fixed pragma version (*0.8.24*) rather than a floating version range. This ensures consistent compilation behavior, improves reproducibility, and prevents unexpected changes from future compiler versions. If compiler updates are needed, they should be explicitly tested and verified before deployment.

Status: Resolved

19. Unused `onlyFailed` modifier should be removed

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:420-423`, the `onlyFailed` modifier is defined but never used.

The modifier ensures that the sale status is `Status.Failed` and reverts with `SaleNotFailed` if not. However, no function in the contract uses this modifier. When the code needs to check for a failed status, it uses inline conditionals instead, such as in `topUpRefunds` and `claimRefund`, which check if `status != Status.Failed && status != Status.Cancelled`.

This dead code increases complexity, maintenance costs, and deployment gas. The `SaleNotFailed` error (line `57`) is also only referenced by this unused modifier, making it effectively unused as well.

Recommendation

We recommend removing the unused `onlyFailed` modifier and `SaleNotFailed` error.

Status: Resolved

20. Using magic numbers decreases code maintainability

Severity: Informational

The contract uses the magic numbers 18 and 1e18 in multiple places to represent the maximum token decimals and normalized token denomination, instead of named constants. This reduces maintainability and increases the risk of inconsistencies if the maximum decimal requirement changes.

The magic number 18 appears in the following locations:

- `packages/evm/contracts/PrimarySale.sol:477 - if (decimals > 18) revert InvalidTokenDecimals();`
- `packages/evm/contracts/PrimarySale.sol:543 - if (decimals == 18) return amount`
- `packages/evm/contracts/PrimarySale.sol:548 - uint256 multiplier = 10 ** (18 - decimals)`
- `packages/evm/contracts/PrimarySale.sol:576 - if (decimals == 18)`
- `packages/evm/contracts/PrimarySale.sol:582 - if (decimals > 18) revert InvalidTokenDecimals()`
- `packages/evm/contracts/PrimarySale.sol:586 - uint256 divisor = 10 ** (18 - decimals)`
- `packages/evm/contracts/PrimarySale.sol:766 - initializeSettlement(): if (assetDecimals > 18)`

Additionally, the literal 1e18 (representing 10^18) is used in six locations for normalization calculations:

- `packages/evm/contracts/PrimarySale.sol:771 - (totalContributedNormalized / MIN_STEP) * 1e18`
- `packages/evm/contracts/PrimarySale.sol:1024 - (contributionNormalized / MIN_STEP) * 1e18`
- `packages/evm/contracts/PrimarySale.sol:1274 - (contribution / MIN_STEP) * 1e18`
- `packages/evm/contracts/PrimarySale.sol:1330 - (totalContributedNormalized / MIN_STEP) * 1e18`
- `packages/evm/contracts/PrimarySale.sol:1446 - (contribution / MIN_STEP) * 1e18`
- `packages/evm/contracts/PrimarySale.sol:1497 - (contributions[i] / MIN_STEP) * 1e18`

While this does not introduce a security vulnerability, using magic numbers reduces code clarity and maintainability. If the maximum decimal requirement changes, all occurrences must be updated manually, increasing the risk of errors and inconsistencies.

Recommendation

We recommend introducing constants `MAX_DECIMALS = 18` and `NORMALIZED_TOKEN_DENOM = 1e18`, for better semantic clarity.

Status: Acknowledged

The client acknowledges this finding, noting that the contract will only be used once.

21. `BatchSettled` event misreports batch numbers when batch sizes differ

Severity: Informational

`packages/evm/contracts/PrimarySale.sol:1001-1076` allows callers to pass any `batchSize` when invoking the settlement loop. After processing a batch, the contract emits `BatchSettled` event, where `batchNumber` is derived as `(settlementState.processedInvestors + batchSize - 1) / batchSize`.

This division assumes that every historical batch uses the current call's `batchSize`, which is not guaranteed. If earlier batches were settled with different sizes, `batchNumber` shifts unpredictably: e.g., after two 100-investor batches (`processedInvestors = 200`), calling the third batch with `batchSize = 50` emits `batchNumber = 4` instead of `3`. Off-chain indexers and monitoring tools relying on `BatchSettled` to track settlement progress or reconcile investor payouts will misinterpret the sequence, potentially skipping batches or double-counting investors even though on-chain accounting is correct.

Recommendation

We recommend tracking the number of completed batches independently from the user-supplied `batchSize`. Maintain a dedicated counter (e.g., `settlementState.processedBatches`) that increments once per successful batch settlement, and emit that counter in `BatchSettled`. Alternatively, change the `BatchSettled` event to emit `startIdx` instead of `batchNumber`.

Status: Resolved

22. Checks-Effects-Interactions pattern not followed in the codebase

Severity: Informational

`packages/evm/contracts/PrimarySale.sol` violates the Check-Effects-Interactions pattern across several flows, most notably in the `invest` function implemented in lines

`634-649`, where `safeTransferFrom` is executed before the investor is registered, contribution totals are updated, or the hard-cap bookkeeping is modified.

Similar early external interactions appear in other pathways:

1. Refunds trigger `safeTransfer` in line `1190` before clearing user balances
2. Settlement pulls asset tokens via `safeTransferFrom` in line `782`, prior to flipping the settlement state
3. Batch settlement pushes tokens to investors using `safeTransfer` in line `1044` before marking distributions
4. Admin recovery and commission flows call `safeTransferFrom` in `827`, `safeTransferFrom` in `1111`, `safeTransferFrom` in `1112`, and in line `1122` ahead of state mutations that depend on those transfers

Despite `nonReentrant` modifier used throughout the codebase, if the external call manages to introduce an unsafe operation, an inconsistent state in the contract may introduce a bug in any possible external contract interactions.

Recommendation

Restructure each affected function to follow strict CEI ordering: perform all validation, then update every relevant piece of contract state (investor arrays, contribution mappings, settlement flags, distribution markers, etc.), and only after state is consistent, execute external calls. When reordering is not possible, switch to a pull pattern so that external transfers occur in separate calls after the state reflects the intended outcome.

Status: Resolved

23. Misleading multisig-specific error in permissionless refund top-up

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:1147-1163`, the `topUpRefunds` function is explicitly permissionless and can be called by any address to replenish the refund pool.

However, when the caller's ERC-20 allowance is insufficient, execution reverts with `InsufficientMultisigAllowance`. The name suggests only the treasury multisig should call this code path, contradicting the notice that "any address can top up refunds," and obscuring the actual cause of failure for third-party callers. This mislabeling misleads operators and monitoring systems, causing unnecessary investigation time or misguided assumptions about access control whenever non-multisig contributors interact with the function.

Recommendation

We recommend renaming the revert to a generic allowance error (e.g., `InsufficientAllowance`) or emitting a context-agnostic message that reflects the permissionless nature of the function.

Status: Acknowledged

The client acknowledges this finding, noting that although the comments are misleading, the function implementation is correct. Furthermore, this function will be used only by the multisig.

24. Repeated base amount calculations decrease code readability

Severity: Informational

`packages/evm/contracts/PrimarySale.sol:771, 1024, 1274, 1330, 1446, 1497` independently recompute `uint256 baseAmount = (value / MIN_STEP) * 1e18` within multiple execution paths (`_finalizeAuction`, `_purchase`, `_settleSingleBid`, `_settleBatch`, etc.). Each instance performs identical arithmetic using the same normalization rule, resulting in duplicate stack operations and increased bytecode size.

Repeating this expression throughout the contract decreases code readability and maintainability. Centralizing the logic into a small internal pure helper both reduces bytecode and allows future changes to the normalization formula to be applied consistently.

Recommendation

We recommend creating an internal pure helper such as `_toBaseAmount(uint256 amount)` that applies `(amount / MIN_STEP) * 1e18` once, and call it from all locations currently duplicating the expression to reduce gas and simplify maintenance.

Status: Acknowledged

The client acknowledges this finding with the note that this contract will only be used once.

25. `DEFAULT_ADMIN_ROLE` is unused, and `ADMIN_ROLE` becomes the de facto owner role

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:519-523`, the constructor only grants `ADMIN_ROLE` to the chosen admin and then sets `ADMIN_ROLE` to be its own admin, but never grants `DEFAULT_ADMIN_ROLE`. The contract effectively ignores `DEFAULT_ADMIN_ROLE` and relies only on `ADMIN_ROLE` for all privileged actions.

This design is technically fine because `ADMIN_ROLE` is granted and can manage itself. However, it breaks the usual OpenZeppelin pattern where `DEFAULT_ADMIN_ROLE` is the top-level owner.

Recommendation

We recommend documenting clearly in specs that `ADMIN_ROLE` is the only admin role and that `DEFAULT_ADMIN_ROLE` is intentionally unused. Alternatively, grant `DEFAULT_ADMIN_ROLE` to the same admin for consistency with common OZ patterns.

Status: Acknowledged

The client acknowledges this finding with the note that the specifications will be updated to reflect the current implementation.

26. `activate` does not check that the current time is before `END`

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:595-600`, the `activate` function only checks that `block.timestamp` is not before `START`, but does not check that it is also not after `END`. This allows the admin to activate a sale after the configured end time, which is inconsistent with the intended sale window enforced elsewhere.

Recommendation

We recommend adding a second check in `activate` to revert when `block.timestamp > END`, so activation is only possible within the configured sale period.

Status: Acknowledged

The client acknowledges this finding, noting that omitting the current time check against `END` in the `activate` function is intentional. This check is not required by design.

27. Commission is not reflected in documented soft cap semantics

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:1108-1113`, the `finalizeSettlement` function computes `tokenCommission = (netAmount * COMMISSION_BPS) / 10_000`, sends this commission to `MANTRA`, and transfers the remaining `tokenIssuerAmount` to `ISSUER`.

However, the soft cap logic in `endSale` and related comments do not document that the issuer's actual proceeds are `SOFT_CAP minus commission`, so when the soft cap is just met, the issuer effectively receives less than `SOFT_CAP` after fees.

Recommendation

We recommend documenting in NATSPEC (and external specifications) that the issuer's net proceeds after commissions are lower than `SOFT_CAP`, and clarifying whether `SOFT_CAP` is intended to apply before or after fees.

Status: Acknowledged

The client acknowledges this finding with the note that, given only one issuer will use this contract, they will communicate the commission behavior directly to that issuer and update the documentation to reflect the soft cap semantics accordingly.

28. `topUpRefunds` design is inconsistent and practically a no-op for external users

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:1147-1163`, the `topUpRefunds` function increases `refundsPoolByToken[token]` and emits `RefundsFunded`, but its behavior has several inconsistencies.

First, it introduces a first-come-first-served race: if the multisig or any funder does not fully top up all tokens, the `claimRefund` function reverts with `InsufficientRefunds` for later callers, so early investors can drain the pool while later ones are blocked.

Second, the NATSPEC states that "any address can top up refunds," but external helpers have no on-chain way to determine which token is short, and `claimRefund` always pays out in the original contributed token, making third-party topping up of arbitrary tokens impractical in practice.

Finally, the explicit allowance check in lines `1154-1157` is redundant because `safeTransferFrom` already reverts on insufficient allowance. Overall, the function is effectively an internal operational tool for the team rather than a reliable, fair mechanism for community-driven top-ups.

Recommendation

We recommend clarifying in NATSPEC and documentation that the `topUpRefunds` function is primarily intended for the multisig or admin to operationally backstop refunds, removing the redundant allowance pre-check, and, if fairness is a goal, redesigning the refund mechanism to avoid first-come-first-served behavior.

Status: Acknowledged

The client acknowledges this finding with the note that the `topUpRefunds` functionality is intended for internal use only.

29. `isSaleActive` ignores the paused state

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:1302-1312`, the `isSaleActive` function checks the status and time window (and hard cap), but does not account for whether the contract is `paused`. As a result, `isSaleActive` can return `true` while the `invest` function is blocked by `whenNotPaused`, which is not consistent.

Recommendation

We recommend extending `isSaleActive` to return `false` when the contract is paused.

Status: Resolved

30. Per-investment forwarding to `MULTISIG` adds overhead and complicates cancellations

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:636-637`, the `invest` function immediately forwards each contribution to `MULTISIG` via `IERC20(token).safeTransfer(MULTISIG, amount)` instead of holding funds in the contract until `endSale`. This design forces the multisig to manage approvals and gas for every later settlement or refund flow. It makes cancelled sales more operationally expensive, since funds are already off the contract and must be actively returned from the multisig.

Recommendation

We recommend considering keeping contributed funds in the `PrimarySale` contract during the sale and only pulling or forwarding them in `endSale`, so that cancellations and settlements can be handled with fewer multisig interactions and lower operational overhead.

Status: Acknowledged

The client acknowledges this finding with the note that per-investment forwarding to `MULTISIG` is required for licensing compliance. They are aware of the overhead and cancellation complexity, but this approach is necessary to meet regulatory and licensing obligations.

31. Outdated comment about miner timestamp manipulation

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:404-407`, the modifier comment states that "miners can manipulate timestamps by ~15 seconds," which reflects pre-merge PoW assumptions and is outdated under the current Ethereum consensus.

Recommendation

We recommend removing the comment.

Status: Resolved

32. Unused `SettlementComplete` event

Severity: Informational

In `packages/evm/contracts/PrimarySale.sol:284-289`, the `SettlementComplete` event is declared but never emitted anywhere in the contract, while `SettlementFinalized` is used instead in the `finalizeSettlement` function. This makes `SettlementComplete` dead code.

Recommendation

We recommend removing the unused event.

Status: Resolved